



第 1 章 Hibernate_day01

今日任务

- 使用 Hibernate 完成对 CRM 系统中客户管理的 DAO 中的 CRUD 的操作

教学导航

教学目标	
教学方法	案例驱动法

案例一：使用 Hibernate 完成 CRM 客户管理的 CRUD 的操作

1.1 案例需求：

1.1.1 需求描述

CRM 系统中客户信息管理模块功能包括：

新增客户信息

客户信息查询

修改客户信息

删除客户信息

1.2 相关知识点：

1.2.1 CRM 的概述

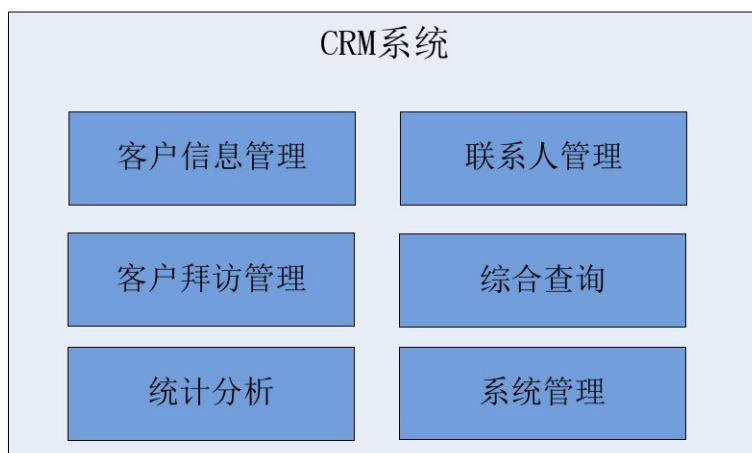
1.2.1.1 什么是 CRM

CRM（Customer Relationship Management）客户关系管理，是利用相应的信息技术以及互联网

技术来协调企业与顾客间在销售、营销和服务上的交互，向客户提供创新式的个性化的客户交互和服务的过程。其最终目标是将面向客户的各项信息和活动集成起来，组建一个以客户为中心的企业，实现对面向客户的活动的全面管理。

1.2.1.2 CRM 的功能模块

CRM 系统实现了对企业销售、营销、服务等各阶段的客户信息、客户活动进行统一管理。CRM 系统功能涵盖企业销售、营销、用户服务等各业务流程，业务流程中与客户相关活动都会在 CRM 系统统一管理，下边列出一些基本的功能模块，包括：客户信息管理、联系人管理、商机管理、统计分析等。



- ✓ 客户信息管理
对客户信息统一维护，客户是指存量客户或拟营销的客户，通过员工录入形成公司的“客户库”是公司最重要的数据资源。
- ✓ 联系人管理
对客户的联系人信息统一管理，联系人是指客户企业的联系人，即企业的业务人员和客户的哪些人在打交道。
- ✓ 客户拜访管理：
业务员（用户）要开发客户需要去拜访客户，客户拜访信息记录了业务员与客户沟通交流方面的不足、采取的策略不当、有待改进的地方或值得分享的沟通技巧等方面的信息。
- ✓ 综合查询
客户相关信息查询，包括：客户信息查询、联系人信息查询、商机信息查询等。
- ✓ 统计分析
按分类统计客户信息，包括：客户信息来源统计、按行业统计客户、客户发展数量统计等。
- ✓ 系统管理
系统管理属于 crm 系统基础功能模块，包括：数据字典、账户管理、角色管理、权限管理、操作日志管理等。

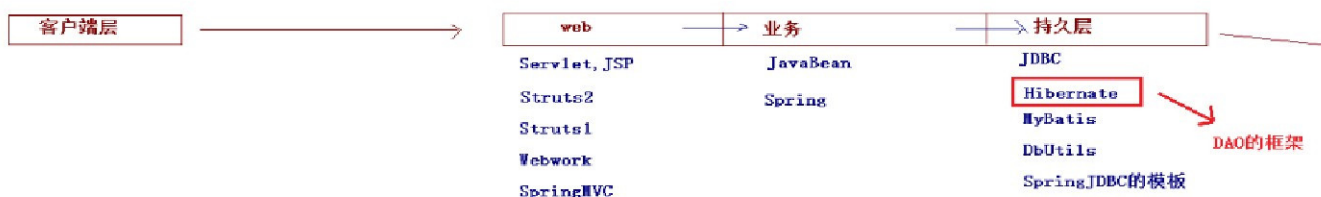


1.2.1.3 JavaEE 开发三层结构

EE开发的三层结构：



EE开发的经典三层结构



1.2.2 Hibernate 的概述

1.2.2.1 什么是 Hibernate

Hibernate 框架是当今主流的 Java 持久层框架之一，由于它具有简单易学、灵活性强、扩展性强等特点，能够大大地简化程序的代码量，提高工作效率，因此受到广大开发人员的喜爱。

Hibernate 是一个开放源代码的 ORM（Object Relational Mapping，对象关系映射）框架，它对 JDBC 进行了轻量级的对象封装，使得 Java 开发人员可以使用面向对象的编程思想来操作数据库。

Hibernate（开放源代码的对象关系映射框架）

编辑

Hibernate是一个开放源代码的对象关系映射框架，它对JDBC进行了非常轻量级的对象封装，它将POJO与数据库表建立映射关系，是一个全自动的orm框架，hibernate可以自动生成SQL语句，自动执行，使得Java程序员可以随心所欲的使用对象编程思维来操纵数据库。Hibernate可以应用在任何使用JDBC的场合，既可以在Java的客户端程序使用，也可以在Servlet/JSP的Web应用中使用，最具革命意义的是，Hibernate可以在应用EJB的J2EE架构中取代CMP，完成数据持久化的重任。

Hibernate 就是一个持久层的 ORM 的框架。

ORM :Object Relational Mapping. 对象关系映射。

1.2.2.2 为什么要学习 Hibernate

使用传统的 JDBC 开发应用系统时，如果是小型应用系统，并不觉得有什么麻烦，但是对于大型应用系统的开发，使用 JDBC 就会显得力不从心。例如对几十、几百张包含几十个字段的表进行插入操作时，编写的 SQL 语句不但很长，而且繁琐，容易出错；在读取数据时，需要写多条 getXXX

语句从结果集中取出各个字段的信息，不但枯燥重复，并且工作量非常大。为了提高数据访问层的编程效率，Gavin King 开发出了一个当今最流行的 ORM 框架，它就是 Hibernate 框架。

所谓的 ORM 就是利用描述对象和数据库表之间映射的元数据，自动把 Java 应用程序中的对象，持久化到关系型数据库的表中。通过操作 Java 对象，就可以完成对数据库表的操作。可以把 ORM 理解为关系型数据和对象的一个纽带，开发人员只需要关注纽带一端映射的对象即可。ORM 原理如图 1-1 所示。



ORM 原理

与其它操作数据库的技术相比，Hibernate 具有以下几点优势：

- Hibernate 对 JDBC 访问数据库的代码做了轻量级封装，大大简化了数据访问层繁琐的重复性代码，并且减少了内存消耗，加快了运行效率。
- Hibernate 是一个基于 JDBC 的主流持久化框架，是一个优秀的 ORM 实现，它很大程度的简化了 DAO（Data Access Object，数据访问对象）层编码工作。
- Hibernate 的性能非常好，映射的灵活性很出色。它支持很多关系型数据库，从一对一到多对多的各种复杂关系。
- 可扩展性强，由于源代码的开源以及 API 的开放，当本身功能不够用时，可以自行编码进行扩展。

1.2.3 Hibernate 的入门:

1.2.3.1 下载 Hibernate5

<http://sourceforge.net/projects/hibernate/files/hibernate-orm/5.0.7.Final/>

Home / hibernate-orm / 5.0.7.Final

Name	Modified	Size	Downloads / Week
↑ Parent folder			
hibernate-release-5.0.7.Final.zip	2016-01-13	94.1 MB	14 ☐
hibernate-release-5.0.7.Final.tgz	2016-01-13	58.4 MB	1 ☐

Hibernate5.0.7 版本下载后，解压完的目录结构如图所示。



这台电脑 ▸ 新加卷 (D:) ▸ hibernate-release-5.0.7.Final			
名称	修改日期	类型	大小
documentation	2016/1/13 12:45	文件夹	
lib	2016/1/13 12:45	文件夹	
project	2016/1/13 12:45	文件夹	
changelog.txt	2016/1/13 12:33	文本文档	46 KB
hibernate_logo.gif	2013/4/17 11:37	GIF 文件	2 KB
lgpl.txt	2013/4/17 11:37	文本文档	26 KB

从图可以看出，hibernate5.0.7 的解压 s 目录中包含一系列的子目录，这些子目录分别用于存放不同功能的文件，接下来针对这些子目录进行简单介绍，具体如下：

- documentation 文件夹：存放 Hibernate 的相关文档，包括参考文档的 API 文档。
- lib 文件夹：存放 Hibernate 编译和运行所依赖的 JAR 包。其中 required 子目录下包含了运行 Hibernate5 项目必须的 JAR 包。
- project 文件夹：存放 Hibernate 各种相关的源代码。

在 lib/required 子目录中，包含的 JAR 包

这台电脑 ▸ 新加卷 (D:) ▸ hibernate-release-5.0.7.Final ▸ lib ▸ required			
名称	修改日期	类型	大小
antlr-2.7.7.jar	2014/4/28 20:30	Executable Jar File	435 KB
dom4j-1.6.1.jar	2014/4/28 20:28	Executable Jar File	307 KB
geronimo-jta_1.1_spec-1.1.1.jar	2015/5/5 11:26	Executable Jar File	16 KB
hibernate-commons-annotations-5.0....	2015/11/30 10:22	Executable Jar File	74 KB
hibernate-core-5.0.7.Final.jar	2016/1/13 12:35	Executable Jar File	5,453 KB
hibernate-jpa-2.1-api-1.0.0.Final.jar	2014/4/28 20:30	Executable Jar File	111 KB
jandex-2.0.0.Final.jar	2015/11/30 10:22	Executable Jar File	184 KB
javassist-3.18.1-GA.jar	2014/4/28 20:28	Executable Jar File	698 KB
jboss-logging-3.3.0.Final.jar	2015/5/28 12:35	Executable Jar File	66 KB

1.2.3.2 创建数据库和表

```
create database hibernate_day01;
use hibernate_day01;
CREATE TABLE `cst_customer` (
  `cust_id` bigint(32) NOT NULL AUTO_INCREMENT COMMENT '客户编号(主键)',
  `cust_name` varchar(32) NOT NULL COMMENT '客户名称(公司名称)',
  `cust_source` varchar(32) DEFAULT NULL COMMENT '客户信息来源',
  `cust_industry` varchar(32) DEFAULT NULL COMMENT '客户所属行业',
  `cust_level` varchar(32) DEFAULT NULL COMMENT '客户级别',
  `cust_phone` varchar(64) DEFAULT NULL COMMENT '固定电话',
  `cust_mobile` varchar(16) DEFAULT NULL COMMENT '移动电话',
```



```
PRIMARY KEY (`cust_id`)  
) ENGINE=InnoDB AUTO_INCREMENT=94 DEFAULT CHARSET=utf8;
```

1.2.3.3 引入 Hibernate 的开发包

数据库驱动包

 mysql-connector-java-5.1.7-bin.jar	2014/7/8 18:41	Executable Jar File	694 KB
--	----------------	---------------------	--------

Hibernate/lib/required/*.jar

 antlr-2.7.7.jar	2014/4/28 20:30	Executable Jar File	435 KB
 dom4j-1.6.1.jar	2014/4/28 20:28	Executable Jar File	307 KB
 geronimo-jta_1.1_spec-1.1.1.jar	2015/5/5 11:26	Executable Jar File	16 KB
 hibernate-commons-annotations-5.0...	2015/11/30 10:22	Executable Jar File	74 KB
 hibernate-core-5.0.7.Final.jar	2016/1/13 12:35	Executable Jar File	5,453 KB
 hibernate-jpa-2.1-api-1.0.0.Final.jar	2014/4/28 20:30	Executable Jar File	111 KB
 jandex-2.0.0.Final.jar	2015/11/30 10:22	Executable Jar File	184 KB
 javassist-3.18.1-GA.jar	2014/4/28 20:28	Executable Jar File	698 KB
 jboss-logging-3.3.0.Final.jar	2015/5/28 12:35	Executable Jar File	66 KB

日志记录的包

 log4j-1.2.16.jar	2015/8/6 14:04	Executable Jar File	471 KB
 slf4j-api-1.6.1.jar	2015/8/6 14:05	Executable Jar File	25 KB
 slf4j-log4j12-1.7.2.jar	2015/8/6 14:05	Executable Jar File	9 KB

1.2.3.4 创建实体（持久化类）

持久化类是应用程序中的业务实体类，这里的持久化是指类的对象能够被持久化保存到数据库中。Hibernate 使用普通 Java 对象（Plain Old Java Object），即 POJO 的编程模式来进行持久化。POJO 类中包含的是与数据库表相对应的各个属性，这些属性通过 getter 和 setter 方法来访问，对外部隐藏了内部的实现细节。下面就来编写 Customer 持久化类。

在项目 src 目录下，创建 cn.itcast.domain 包，并在包中创建实体类 Customer（对应数据库表 cst_customer），Customer 类包含与 cst_customer 数据表字段对应的属性，以及相应的 getXxx ()和 setXxx ()方法。

```
public class Customer {  
    private Long cust_id;  
    private String cust_name;  
    private String cust_source;  
    private String cust_industry;  
    private String cust_level;  
    private String cust_phone;  
    private String cust_mobile;
```



```
public Long getCust_id() {  
    return cust_id;  
}  
  
public void setCust_id(Long cust_id) {  
    this.cust_id = cust_id;  
}  
  
public String getCust_name() {  
    return cust_name;  
}  
  
public void setCust_name(String cust_name) {  
    this.cust_name = cust_name;  
}  
  
public String getCust_source() {  
    return cust_source;  
}  
  
public void setCust_source(String cust_source) {  
    this.cust_source = cust_source;  
}  
  
public String getCust_industry() {  
    return cust_industry;  
}  
  
public void setCust_industry(String cust_industry) {  
    this.cust_industry = cust_industry;  
}  
  
public String getCust_level() {  
    return cust_level;  
}  
  
public void setCust_level(String cust_level) {  
    this.cust_level = cust_level;  
}  
  
public String getCust_phone() {  
    return cust_phone;  
}  
  
public void setCust_phone(String cust_phone) {  
    this.cust_phone = cust_phone;  
}  
  
public String getCust_mobile() {  
    return cust_mobile;  
}  
  
public void setCust_mobile(String cust_mobile) {  
    this.cust_mobile = cust_mobile;  
}  
  
}
```




1.2.3.5 创建映射文件

实体类 Customer 目前还不具备持久化操作的能力，而 Hibernate 需要知道实体类 Customer 映射到数据库 Hibernate 中的哪个表，以及类中的哪个属性对应数据库表中的哪个字段，这些都需要在映射文件中配置。

在实体类 Customer 所在的包中，创建一个名称为 Customer.hbm.xml 的映射文件，在该文件中定义了实体类 Customer 的属性是如何映射到 cst_customer 表的列上的。

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-mapping PUBLIC
    "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
    "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd">
<hibernate-mapping>
    <!-- 建立类和表的一个映射关系: -->
    <!--
        class 标签:用来建立类和表的映射
        * name 属性   : 类中的全路径.
        * table 属性   : 表名(如果类名和表名是一致的, 那么 table 属性客户省略)
        * catalog 属性: 数据库名称, 可以省略
    -->
    <class name="cn.itcast.domain.Customer" table="cst_customer">
        <!-- 建立类中的属性与表中的主键的映射 -->
        <!--
            id 标签: 用来建立类中的属性与表中的主键字段对应
            * name 属性   : 类中的属性名
            * column 属性   : 表中字段名(如果类中的属性名和表中的字段名一致, 那么省略
column)
            * length 属性   : 字段的长度.
            * type 属性   : 类型. 写 Java 数据类型, Hibernate 数据类型(默认), SQL 类型
        -->
        <id name="cust_id" column="cust_id">
            <!-- 主键生成策略 -->
            <generator class="native"/>
        </id>

        <!-- 建立类中的普通属性与表中的字段的映射 -->
        <!--
            property 标签: 用来建立类中的普通属性与表中的字段对应
            * name 属性   : 类中的属性名
            * column 属性   : 表中字段名(如果类中的属性名和表中的字段名一致, 那么省略
column)
            * length 属性   : 字段的长度.
            * type 属性   : 类型. 写 Java 数据类型, Hibernate 数据类型(默认), SQL 类型
        -->
```




```
<property name="cust_name" column="cust_name"/>
<property name="cust_source" column="cust_source"/>
<property name="cust_industry" column="cust_industry"/>
<property name="cust_level" column="cust_level"/>
<property name="cust_phone" column="cust_phone"/>
<property name="cust_mobile" column="cust_mobile"/>
</class>
</hibernate-mapping>
```

1.2.3.6 创建 Hibernate 的核心配置文件

Hibernate 的映射文件反映了持久化类和数据库表的映射信息，而 Hibernate 的配置文件则主要用来配置数据库连接以及 Hibernate 运行时所需要的各个属性的值。在项目的 src 下创建一个名称为 hibernate.cfg.xml 的文件

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC
    "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
    "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
<hibernate-configuration>
    <session-factory>
        <!-- 必要的配置信息:连接数据库的基本参数 -->
        <property
name="hibernate.connection.driver_class">com.mysql.jdbc.Driver</property>
        <property
name="hibernate.connection.url">jdbc:mysql:///hibernate_day01</property>
        <property name="hibernate.connection.username">root</property>
        <property name="hibernate.connection.password">123</property>

        <!-- Hibernate 的属性 -->
        <!-- Hibernate 的方言:作用, 根据配置的方言生成相应的 SQL 语句 -->
        <property
name="hibernate.dialect">org.hibernate.dialect.MySQLDialect</property>

        <!-- Hibernate 显示 SQL 语句: -->
        <property name="hibernate.show_sql">true</property>
        <!-- Hibernate 格式化 SQL 语句: -->
        <property name="hibernate.format_sql">true</property>
        <!-- Hibernate 的 hbm2ddl (数据定义语言:create drop alter ...) 属性 -->
        <!--
            hbm2ddl.auto 的取值
            * none          :不用 Hibernate 自动生成表.
            * create        :每次都会创建一个新的表.(测试)
            * create-drop    :每次都会创建一个新的表, 执行程序结束后删除这个表.(测
```



试)

* update :如果数据库中有表,使用原来的表,如果没有表,创建一个新表.可以更新表结构。

* validate :只会使用原有的表.对映射关系进行校验.

-->

```
<property name="hibernate.hbm2ddl.auto">update</property>
```

```
<!-- Hibernate 加载映射 -->
```

```
<mapping resource="cn/itcast/domain/Customer.hbm.xml"/>
```

```
</session-factory>
```

```
</hibernate-configuration>
```

该配置文件设置了数据库连接的相关属性以及其它的一些常用属性,并且通过 mapping 的 resource 属性将对象的映射信息加入到了 Hibernate 的配置文件中。

1.2.3.7 编写测试代码

在项目中新建一个名称为 cn.itcast.test 的包,然后在包中建立一个名为 HibernateDemo1.java 的文件,该文件是用来测试的类文件。

```
public class HibernateDemo1 {

    @Test
    /**
     * 使用 Hibernate 保存数据
     */
    public void demo1(){
        // 1.加载配置文件:
        Configuration cfg = new Configuration().configure();
        // 2.创建一个 SessionFactory:
        SessionFactory sessionFactory = cfg.buildSessionFactory();
        // 3.创建 Session 对象.Session 对象 类似 Connection.
        Session session = sessionFactory.openSession();
        // 4.开启事务:
        Transaction tx = session.beginTransaction();
        // 5.执行相关操作
        Customer customer = new Customer();
        customer.setCust_name("小王");
        customer.setCust_source("网络推广");

        session.save(customer);
        // 6.事务提交
        tx.commit();
        // 7.释放资源
        session.close();
    }
}
```



```
}  
}
```

首先创建 Configuration 类的实例，并通过它来读取并解析配置文件 hibernate.cfg.xml。然后创建 SessionFactory 读取解析映射文件信息，并将 Configuration 对象中的所有配置信息拷贝到 SessionFactory 内存中。接下来，打开 Session，让 SessionFactory 提供连接，并开启一个事务，之后创建对象，向对象中添加数据，通过 session.save()方法完成向数据库中保存数据的操作。最后提交事务，并关闭资源。

1.2.4 Hibernate 的常见配置:

在案例中，已经接触过 Hibernate 的映射文件和配置文件。接下来，将对这些文件进行详细的讲解。

1.2.4.1 映射文件的配置

该文件用于向 Hibernate 提供持久化类到关系型数据库的映射，每个映射文件的的结构基本都是相同的，其普遍的代码形式如下所示。

```
<?xml version="1.0" encoding="UTF-8"?>  
<!--映射文件的 dtd 信息 -->  
<!DOCTYPE hibernate-mapping PUBLIC  
    "-//Hibernate/Hibernate Mapping DTD 3.0//EN"  
    "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd">  
<hibernate-mapping>  
    <!-- name 代表的是实体类名 table 代表的是表名 -->  
    <class name="XXX" table="xxx">  
        <!-- name=id 代表的是 XXX 类中属性 column=id 代表的是 xxx 表中的字段 -->  
        <id name="id" column="id">  
            <generator class="native"/><!-- 主键生成策略 -->  
        </id>  
        <!-- 其它属性使用 property 标签来映射 -->  
        <property name="XXX" column="xxx" type="string" />  
    </class>  
</hibernate-mapping>
```

映射文件通常是一个 XML 文件即可,但一般命名为类名.hbm.xml

【<class>标签】



```
<hibernate-mapping>
    <!-- 建立类和表的一个映射关系： -->
    <!--
        class标签：用来建立类和表的映射
        * name属性    ：类中的全路径。
        * table属性    ：表名（如果类名和表名是一致的，那么table属性客户省略）
        * catalog属性：数据库名称，可以省略
    -->
    <class name="cn.itcast.domain.Customer" table="cst_customer">
```

【<id>标签】

```
<!-- 建立类中的属性与表中的主键的映射 -->
<!--
    id标签：用来建立类中的属性与表中的主键字段对应
    * name属性：类中的属性名
    * column属性：表中字段名（如果类中的属性名和表中的字段名一致，那么省略column）
    * length属性：字段的长度。
    * type属性：类型。写Java数据类型，Hibernate数据类型（默认），SQL类型
-->
<id name="cust_id" column="cust_id">
    <!-- 主键生成策略 -->
    <generator class="native"/>
</id>
```

【<property>标签】

```
<!-- 建立类中的普通属性与表中的字段的映射 -->
<!--
    property标签：用来建立类中的普通属性与表中的字段对应
    * name属性    ：类中的属性名
    * column属性  ：表中字段名（如果类中的属性名和表中的字段名一致，那么省略column）
    * length属性  ：字段的长度。
    * type属性    ：类型。写Java数据类型，Hibernate数据类型（默认），SQL类型
-->
<property name="cust_name" column="cust_name"/>
<property name="cust_source" column="cust_source"/>
<property name="cust_industry" column="cust_industry"/>
<property name="cust_level" column="cust_level"/>
<property name="cust_phone" column="cust_phone"/>
<property name="cust_mobile" column="cust_mobile"/>
```

1.2.4.2 核心配置

Hibernate 的配置文件，包含了连接持久层与映射文件所需的基本信息，其配置文件有两种格式，具体如下：

- 一种是 properties 属性文件格式的配置文件，它使用键值对的形式存放信息，默认文件名称为 hibernate.properties；
- 另一种是 XML 格式的配置文件，XML 配置文件的默认名称为 hibernate.cfg.xml。

上述两种格式的配置文件是等价的，具体使用哪个可以自由选择。XML 格式的配置文件更易于



修改，配置能力更强，当改变底层应用配置时不需要改变和重新编译代码，只修改配置文件的相应属性即可，而 properties 格式的文件则不具有此优势，因此，在实际开发项目中，大多数情况会使用 XML 格式的配置文件。下面将对 XML 格式的配置文件进行详细介绍。

hibernate.cfg.xml 配置文件一般在开发时会放置在 src 的源文件夹下，发布后，该文件会在项目的 WEB-INF/classes 路径下。配置文件的常用配置信息如下所示。

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC
    "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
    "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
<hibernate-configuration>
    <session-factory>
        <!-- 必要的配置信息:连接数据库的基本参数 -->
        <property
name="hibernate.connection.driver_class">com.mysql.jdbc.Driver</property>
        <property
name="hibernate.connection.url">jdbc:mysql:///hibernate_day01</property>
        <property name="hibernate.connection.username">root</property>
        <property name="hibernate.connection.password">123</property>

        <!-- Hibernate 的属性 -->
        <!-- Hibernate 的方言:作用, 根据配置的方言生成相应的 SQL 语句 -->
        <property
name="hibernate.dialect">org.hibernate.dialect.MySQLDialect</property>

        <!-- Hibernate 显示 SQL 语句: -->
        <property name="hibernate.show_sql">true</property>
        <!-- Hibernate 格式化 SQL 语句: -->
        <property name="hibernate.format_sql">true</property>
        <!-- Hibernate 的 hbm2ddl(数据定义语言:create drop alter ...)属性 -->
        <!--
            hbm2ddl.auto 的取值
                * none          :不用 Hibernate 自动生成表.
                * create        :每次都会创建一个新的表.(测试)
                * create-drop    :每次都会创建一个新的表, 执行程序结束后删除这个表.(测试)
                * update         :如果数据库中有表, 使用原来的表, 如果没有表, 创建一个新表. 可以更新表结构.
                * validate       :只会使用原有的表. 对映射关系进行校验.
            -->
        <property name="hibernate.hbm2ddl.auto">update</property>

        <!-- Hibernate 加载映射 -->
        <mapping resource="cn/itcast/domain/Customer.hbm.xml"/>
    </session-factory>
```

```
</hibernate-configuration>
```

在上述代码中，首先进行了 xml 声明，然后是配置文件的 dtd 信息，该信息同样可以在核心包 hibernate-core-5.0.7.Final.jar 下的 org.hibernate 包中的 hibernate-configuration-3.0.dtd 文件中找到，读者只需要复制过来用即可，不需要刻意记忆。

Hibernate 配置文件的根元素是 hibernate-configuration，该元素包含子元素 session-factory，在 session-factory 元素中又包含多个 property 元素，这些 property 元素用来对 Hibernate 连接数据库的一些重要信息进行配置。例如，上面的配置文件中，使用了 property 元素配置了数据库的方言、驱动、URL、用户名、密码等信息。最后通过 mapping 元素的配置，加载出映射文件的信息。

Hibernate 配置文件的一些常用属性名称及用途，如表所示。

表1-1 Hibernate 常用配置属性

名称	用途
hibernate.dialect	操作数据库方言
hibernate.connection.driver_class	连接数据库驱动程序
hibernate.connection.url	连接数据库 URL
hibernate.connection.username	数据库用户名
hibernate.connection.password	数据库密码
hibernate.show_sql	在控制台上输出 SQL 语句
hibernate.format_sql	格式化控制台输出的 SQL 语句
hibernate.hbm2ddl.auto	当 SessionFactory 创建时是否根据映射文件自动验证表结构或自动创建、自动更新数据库表结构。该参数的取值为：validate、update、create 和 create-drop。
hibernate.connection.autocommit	事务是否自动提交

1.2.5 Hibernate 的相关 API:

1.2.5.1 Configuration:配置对象.

Configuration 主要用于 Hibernate 框架加载映射文件

Configuration

Configuration 类的作用是对 Hibernate 进行配置，以及对它进行启动。在 Hibernate 的启动过程中，Configuration 类的实例首先定位映射文档的位置，读取这些配置，然后创建一个 SessionFactory 对象。虽然 Configuration 类在整个 Hibernate 项目中只扮演着一个很小的角色，但它是启动 hibernate 时所遇到的第一个对象。

【加载核心配置文件】

在使用 Hibernate 时，首先要创建 Configuration 实例，Configuration 实例主要用于启动、加载、管理 hibernate 的配置文件信息。在启动 Hibernate 的过程中，Configuration 实例首先确定 Hibernate 配置文件的位置，然后读取相关配置，最后创建一个唯一的 SessionFactory 实例。Configuration 对象只存在于系统的初始化阶段，它将 SessionFactory 创建完成后，就完成了自己的使命。

Hibernate 通常使用 Configuration config = new Configuration().configure(); 的方式创建实例，此种方式默认会去 src 下读取 hibernate.cfg.xml 配置文件。如果不想使用默认的 hibernate.cfg.xml 配置文件，而是使用指定目录下（或自定义）的配置文件，则需要向 configure() 方法中传递一个文件路径的参数，其代码写法如下：



```
Configuration config = new Configuration().configure("xml 文件位置");
```

此种写法 hibernate 会去指定位置查找配置文件，例如，想要使用 src 下 config 包中的 hibernate.cfg.xml 文件，只需将文件位置加入 configure()中即可，其代码如下所示：

```
Configuration config = new Configuration().configure("/config/hibernate.cfg.xml");
```

【加载映射文件】

Hibernate 除了可以使用 Configuration 对象加载核心配置文件以外，还可以利用该对象加载映射文件。因为如何使用 properties 文件作为 Hibernate 的核心配置文件，其他的属性可以使用 key=value 的格式来设置，但是映射没有办法加载。这时这个对象就有了用武之地。可以在手动编写代码的时候去加载映射文件。

```
Configuration configuration = new Configuration().configure("xml 文件位置");  
configuration.addResource("cn/itcast/domain/Customer.hbm.xml");
```

1.2.5.2 SessionFactory：Session 工厂对象

SessionFactory

SessionFactory 接口负责初始化 Hibernate。它充当数据存储源的代理，并负责创建 Session 对象。这里用到了工厂模式。需要注意的是 SessionFactory 并不是轻量级的，因为一般情况下，一个项目通常只需要一个 SessionFactory 就够，当需要操作多个数据库时，可以为每个数据库指定一个 SessionFactory。

SessionFactory 接口负责 Hibernate 的初始化和建立 Session 对象。它在 Hibernate 中起到一个缓冲区作用，Hibernate 可以将自动生成的 SQL 语句、映射数据以及某些可重复利用的数据放在这个缓冲区中。同时它还保存了对数据库配置的所有映射关系，维护了当前的二级缓存。

SessionFactory 实例是通过 Configuration 对象获取的，其获取方法如下所示。

```
SessionFactory sessionFactory = config.buildSessionFactory();
```

SessionFactory 具有以下特点：

- 它是线程安全的，它的同一个实例能够供多个线程共享。
- 它是重量级的，不能随意的创建和销毁它的实例。

由于 SessionFactory 的这些特点，一般情况下，一个项目中只需要一个 SessionFactory，只有当应用中存在多个数据源时，才为每个数据源建立一个 SessionFactory 实例。因此，在实际项目使用中，通常会抽取出一个 HibernateUtils 的工具类，用来提供 Session 对象。

Hibernate 的工具类：

```
public class HibernateUtils {  
    private static final Configuration configuration;  
    private static final SessionFactory sessionFactory;  
  
    static{  
        configuration = new Configuration().configure();  
        sessionFactory = configuration.buildSessionFactory();  
    }  
  
    /**  
     * 提供获得 session 的方法：  
     */  
}
```




```
public static Session openSession(){
    return sessionFactory.openSession();
}
}
```

SessionFactory 内部还维护了一个连接池，如果我们需要使用第三方的连接池如 C3P0，那么需要我们自己手动进行配置

配置 C3P0 内容如下

引入 c3p0 的相关 jar 包：

 c3p0-0.9.2.1.jar	2014/4/28 20:30	Executable Jar File	414 KB
 hibernate-c3p0-5.0.7.Final.jar	2016/1/13 12:42	Executable Jar File	12 KB
 mchange-commons-java-0.2.3.4.jar	2014/4/28 20:30	Executable Jar File	568 KB

```
<!-- 配置 C3P0 连接池 -->
<property
name="connection.provider_class">org.hibernate.connection.C3P0ConnectionProvider</p
roperty>
<!--在连接池中可用的数据库连接的最少数目 -->
<property name="c3p0.min_size">5</property>
<!--在连接池中所有数据库连接的最大数目 -->
<property name="c3p0.max_size">20</property>
<!--设定数据库连接的过期时间,以秒为单位,
<!--如果连接池中的某个数据库连接处于空闲状态的时间超过了 timeout 时间,就会从连接池中清除 -->
<property name="c3p0.timeout">120</property>
<!--每 3000 秒检查所有连接池中的空闲连接 以秒为单位-->
<property name="c3p0.idle_test_period">3000</property>
```

1.2.5.3 Session :

Session

Session 接口负责执行被持久化对象的 CRUD 操作(CRUD 的任务是完成与数据库的交流，包含了很多常见的 SQL 语句)。但需要注意的是 Session 对象是非线程安全的。同时，Hibernate 的 session 不同于 JSP 应用中的 HttpSession。这里当使用 session 这个术语时，其实指的是 Hibernate 中的 session，而以后会将 HttpSession 对象称为用户 session。

Session 是应用程序与数据库之间交互操作的一个单线程对象，是 Hibernate 运作的中心，它的主要功能是为持久化对象提供创建、读取和删除的能力，所有持久化对象必须在 session 的管理下才可以进行持久化操作。

创建 SessionFactory 实例后，就可以通过它获取 Session 实例。获取 Session 实例有两种方式，一种是通过 openSession() 方法，另一种是通过 getCurrentSession() 方法。两种方法获取 session 的代码如下所示：

```
//采用 openSession 方法创建 session
Session session = sessionFactory.openSession();
//采用 getCurrentSession() 方法创建 session
```




```
Session session = sessionFactory.getCurrentSession();
```

以上两种获取 session 实例方式的主要区别是，采用 openSession 方法获取 Session 实例时，SessionFactory 直接创建一个新的 Session 实例，并且在使用完成后需要调用 close 方法进行手动关闭。而 getCurrentSession 方法创建的 Session 实例会被绑定到当前线程中，它在提交或回滚操作时会自动关闭。

Session 是线程不安全的，多个并发线程同时操作一个 Session 实例时，就可能导致 Session 数据存取的混乱（方法内部定义和使用 Session 时，不会出现线程问题）。因此设计软件架构时，应避免多个线程共享一个 Session 实例。同时它也是轻量级的，实例的创建和销毁不需要消耗太多的资源。它还有一个缓存，即 Hibernate 的一级缓存，这个缓存主要用于存放当前工作单元加载的对象。

在 Session 中提供了大量的常用方法，具体如下：

- save()、update() 和 saveOrUpdate()方法：用于增加和修改对象。
- delete()方法：用于删除对象。
- get()和 load()方法：根据主键查询。
- createQuery() 和 createSQLQuery()方法：用于数据库操作对象。
- createCriteria()方法：条件查询。

1.2.5.4 Transaction

Transaction

Transaction 接口是一个可选的API，可以选择不使用这个接口，取而代之的是Hibernate 的设计者自己写的底层事务处理代码。Transaction 接口是对实际事务实现的一个抽象，这些实现包括JDBC的事务、JTA 中的UserTransaction、甚至可以是CORBA 事务。之所以这样设计是能让开发者能够使用一个统一事务的操作界面，使得自己的项目可以在不同的环境和容器之间方便地移植。

Transaction 接口主要用于管理事务，它是 Hibernate 的数据库事务接口，且对底层的事务接口进行了封装。Transaction 接口的事务对象是通过 Session 对象开启的，其开启方式如下所示。

```
Transaction transaction = session.beginTransaction();
```

在 Transaction 接口中，提供了事务管理的常用方法，具体如下：

- commit()方法：提交相关联的 session 实例。
- rollback()方法：撤销事务操作。

Session 执行完数据库操作后，要使用 Transaction 接口的 commit()方法进行事务提交，才能真正的将数据操作同步到数据库中。发生异常时，需要使用 rollback()方法进行事务回滚，以避免数据发生错误。因此，在持久化操作后，必须调用 Transaction 接口的 commit()方法和 rollback()方法。如果没有开启事务，那么每个 Session 的操作，都相当于一个独立的操作。